

CC Report

LL(1) Parsing

M. Abdullah Siddiqui | 23I-0617 | F

Moiz Ansari | 23I-0523 | F

Introduction

LL(1) parsing is a top-down parsing technique used in compiler design to analyze whether a given input string conforms to a specified context-free grammar. The term “LL(1)” stands for scanning the input from Left to right, producing a Leftmost derivation, and using 1 lookahead symbol to make parsing decisions.

In LL(1) parsing, a parsing table is constructed using the FIRST and FOLLOW sets of the grammar. This table guides the parser in selecting the correct production rule based on the current non-terminal and the next input symbol. Because it relies on a single lookahead token, LL(1) parsing is efficient and deterministic, making it suitable for real-time syntax analysis.

However, not all grammars are compatible with LL(1) parsing. The grammar must be free of left recursion and should be left-factored to avoid ambiguity and conflicts in the parsing table. When these conditions are satisfied, LL(1) parsing provides a simple and effective method for building predictive parsers.

Approach

Data Structures Used

Grammar, GrammarRule, Production structs for the storage of CFGs. Basically the following structure Grammar contains Grammar Rules+non terminals+terminals, Grammar Rules contain Productions, Productions contain Symbols.

Data Type	Data Structure
Grammar	struct
GrammarRules	map<NT(string), GrammarRule struct>
Productions	vector<Production>
Symbols	vector<string>
NonTerminals	vector<string>
Terminals	vector<string>
First and Follow Sets	map<NT(string), set<Terminals(string)>>

LL1 Parse Table	map<NT, map<T, grammarRule>>
TreeNode	struct

Algorithm Implementation Details

Left Recursion

The algorithm eliminates both direct and indirect left recursion from a context-free grammar to make it suitable for LL(1) parsing.

It processes each non-terminal one by one. For every non-terminal, the productions are first divided into two groups: alpha productions (those that begin with the same non-terminal, representing direct left recursion) and beta productions (those that do not start with the non-terminal).

Before handling direct recursion, the algorithm removes indirect left recursion by substituting productions of previously processed non-terminals wherever they appear at the beginning of a production. This ensures that hidden recursion through other non-terminals is eliminated.

If no direct left recursion exists, the beta productions are copied as they are. However, if direct left recursion is found, a new non-terminal is introduced. The beta productions are modified to append this new non-terminal, while alpha productions are transformed by removing the recursive part and appending the new non-terminal. An epsilon production is also added to allow termination.

Finally, the transformed productions are added to a new grammar, resulting in an equivalent grammar that is free of left recursion and suitable for predictive parsing.

Left Factoring

The left factoring algorithm transforms a grammar to remove common prefixes from productions, making it suitable for LL(1) parsing.

The algorithm iteratively processes each non-terminal in the grammar. For every non-terminal, its productions are first sorted so that productions with the same starting symbol are grouped together. It then scans the productions to detect common prefixes among them.

When two or more productions share the same initial symbol, a new non-terminal is introduced. The common prefix is factored out and retained in the original non-terminal, while the remaining

parts (suffixes) of those productions are moved to the new non-terminal. If any production becomes empty after removing the prefix, it is replaced with an epsilon production.

This process is repeated until no further changes occur, ensuring that all common prefixes are eliminated. The resulting grammar has uniquely distinguishable productions, allowing the parser to make decisions based on a single lookahead symbol.

First Sets

For a terminal or epsilon ("epsilon"), the FIRST set is the symbol itself. For a non-terminal, recursively compute FIRST sets of its productions. For each production, examine symbols from left to right. Add all terminals from the FIRST set of the first symbol. If the first symbol can derive epsilon, continue to the next symbol. If all symbols in a production can derive epsilon, include epsilon in the FIRST set of the non-terminal. Repeat until all non-terminals have stable FIRST sets.

Functions in Code:

computeFirstOfSymbol : computes FIRST of a single symbol.

computeFirstOfString : computes FIRST of a sequence of symbols.

computeFirstOfNT : computes FIRST of a non-terminal.

computeFirst : computes FIRST sets for all non-terminals in the grammar.

Follow Set

Initialize FOLLOW sets for all non-terminals as empty. Add \$ (end of input) to the FOLLOW set of the start symbol. For each production in the grammar, scan its symbols. If a non-terminal A is followed by a terminal, add that terminal to FOLLOW(A). If followed by a non-terminal B, add all symbols in FIRST(B) except epsilon to FOLLOW(A). If B can derive epsilon, or A is at the end, add FOLLOW(parent non-terminal) to FOLLOW(A). Repeat until FOLLOW sets do not change in an iteration (fixed-point algorithm).

Parse Table

This function constructs the LL(1) parsing table for a given grammar using its FIRST and FOLLOW sets. It iterates over all non-terminals, initializing each entry in the table for every terminal and the end-of-input symbol `\$` with an empty production. For each production of a non-terminal, it computes the FIRST set of that production. If a terminal in the FIRST set is not epsilon, the production is placed in the corresponding table cell for that terminal. If the FIRST set contains epsilon, the production is placed in all table cells corresponding to the terminals in the FOLLOW set of the non-terminal. The code also checks for conflicts, printing an error if a table cell already



contains a production, which indicates the grammar may not be LL(1). This table allows a predictive parser to select the correct production based on the current non-terminal and the next input symbol.

Design Decisions

In designing the LL(1) parser and the associated grammar preprocessing tools, several key decisions were made to ensure correctness, efficiency, and clarity. First, the grammar is represented using structured classes for non-terminals, terminals, productions, and rules, which allows easy manipulation and traversal. To handle left recursion, both direct and indirect forms are addressed separately: indirect recursion is removed first by substituting productions of earlier non-terminals, followed by elimination of direct left recursion through the introduction of new non-terminals and epsilon productions. Left factoring is applied iteratively to remove common prefixes from productions, ensuring that each non-terminal's options are distinguishable with a single lookahead symbol. FIRST and FOLLOW sets are computed using recursive and fixed-point algorithms, carefully handling epsilon derivations to guarantee accurate table construction. The LL(1) parsing table is built as a nested map from non-terminals to terminals, allowing efficient lookup during parsing while also detecting conflicts that indicate non-LL(1) grammars. Finally, the parse tree is represented as a tree structure, and an ASCII-based printing scheme was chosen to ensure clear and portable visualization across different environments, avoiding Unicode rendering issues. Overall, these design decisions prioritize modularity, correctness, and ease of debugging, making the parser robust and extensible.

Handling Indirect Left Recursion

The approach to handling indirect left recursion involves systematically eliminating cases where a non-terminal produces another non-terminal that eventually leads back to itself, creating a hidden recursion through other symbols. The algorithm processes the grammar's non-terminals in order and, for each non-terminal, examines its productions. If a production begins with a previously processed non-terminal, the algorithm substitutes that non-terminal's productions into the current production, effectively "inlining" its rules. This transformation converts indirect recursion into direct recursion, which can then be eliminated using standard left recursion removal techniques. By repeating this process for all non-terminals, the grammar is transformed into an equivalent form that is free of both direct and indirect left recursion, making it suitable for LL(1) parsing.

Error Recovery Method

The error recovery was done during the parsing of input string in the same function on the following criteria:

- Stack is empty, but input still has tokens, it means there is garbage at the end, so the parsing is aborted at that instant and error is displayed.
- If a terminal on top of stack does not match with the lookahead, error is displayed, the input pointer is moved on and parsing is continued.
- If the top of stack has a non-terminal and lookahead exists, but the production for the corresponding rule is empty, then the error is displayed, the non-terminal is popped out of stack and parsing is continued.

Challenges Faced

This assignment was pretty easy to divide up among team members and incremental development procedure. However, dealing with text based operations was difficult. We had to remove left recursion, make left factoring, and generate parse tables in the form of code, which was a difficult task to achieve in CPP while considering no memory leaks and inbound access. Further Ramzan schedule made it difficult to work on lengthy code operations. Also checking the correctness of our code was difficult.

Test Cases

First & Follow Test Cases

1. Start -> Expr Tail

Tail -> + Expr Tail | epsilon

Expr -> Term Opt

Opt -> * Term Opt | epsilon

Term -> id | (Start) | epsilon

2. Start -> (Low) | a

Low -> Start Low1

Low1 -> , Start Low1 | epsilon

3. Start -> a Bus Daisy h

Bus -> c Cry

Cry $\rightarrow$ b Cry | epsilon

Daisy $\rightarrow$ End Fuel

End $\rightarrow$ g | epsilon

Fuel $\rightarrow$ f | epsilon

Indirect Left Recursion

1. Expr $\rightarrow$ Term + Term | Term

Term $\rightarrow$ Expr * Factor | Factor

Factor $\rightarrow$ (Expr) | id

Left Factoring

1. Stmt $\rightarrow$ if Cond then Stmt | if Cond then Stmt else Stmt | read id | write id

Cond $\rightarrow$ true | false

Parsing Testing

Grammar

E $\rightarrow$ T E'

E' $\rightarrow$ + T E' | epsilon

T $\rightarrow$ F T'

T' $\rightarrow$ * F T' | epsilon

F $\rightarrow$ (E) | id

- **Invalid Input (Error Handling)**

id + (id * id

id + * id

id +

id id

* id + id

((id + id)

- **Invalid Input (Error Handling)**

id + (id * id)

id + id * id

id * id + id

(id * id) + id

id * id

Verification

Already Defined Solutions

<https://www.gatevidyalay.com/left-recursion-left-recursion-elimination/>

The above-mentioned website contained a few tricky grammars to be checked for recursion of any type.

Lecture Studied Grammars

Grammars that we practiced in the class for LL1 parsing were used.

Use of Generative AI

Sample grammars were given to chatGPT to check for required answers. However this method was not very accurate so we did not rely on it.

Self Solving

We solved the grammars by hand to check if our solutions match the result or not.

Sample Outputs

Error Handling Output

```
=== Original Grammar ===
E -> T E'
E' -> + T E' | epsilon
T -> F T'
T' -> * F T' | epsilon
F -> ( E ) | id

=== Left Recursion Removed Grammar ===
E -> T E'
E' -> + T E' | epsilon
T -> F T'
T' -> * F T' | epsilon
F -> ( E ) | id

=== Left Factored Grammar ===
E -> T E'
E' -> + T E' | epsilon
T -> F T'
T' -> * F T' | epsilon
F -> ( E ) | id

[Success] Grammar safely saved to output/grammar_transformed.txt

=== First & Follow Sets ===
--- First() Sets ---
E:      { (, id }
E':     { +, epsilon }
T:      { (, id }
T':     { *, epsilon }
F:      { (, id }

--- Follow() Sets ---
E:      { $, ) }
E':     { $, ) }
T:      { $, ), + }
T':     { $, ), + }
F:      { $, ), *, + }

===== LL(1) PARSE TABLE =====
      +      *      (      )      id      $
-----
E      -      -      E->TE'      -      E->TE'      -
E'     E'->+TE'      -      E'->epsilon      -      E'->epsilon      -
T      -      -      T->FT'      -      T->FT'      -
T'     T'->epsilon      T'->*FT'      -      T'->epsilon      -      T'->epsilon      -
F      -      -      F->(E)      -      F->id      -

[Success] Parsing table safely saved to output/parsing_table.txt
```

```

===== PARSING TRACE 1 =====
Step | Stack | Input | Action
-----|-----|-----|-----
1 | $ E | id + ( id * id $ | Gen E -> T E'
2 | $ E' T | id + ( id * id $ | Gen T -> F T'
3 | $ E' T' F | id + ( id * id $ | Gen F -> id
4 | $ E' T' id | + ( id * id $ | Match id
5 | $ E' T' | + ( id * id $ | Gen T' -> epsilon
6 | $ E' | + ( id * id $ | Gen E' -> + T E'
7 | $ E' T + | ( id * id $ | Match +
8 | $ E' T | ( id * id $ | Gen T -> F T'
9 | $ E' T' F | ( id * id $ | Gen F -> ( E )
10 | $ E' T' ) E ( | id * id $ | Match (
11 | $ E' T' ) E | id * id $ | Gen E -> T E'
12 | $ E' T' ) E' T | id * id $ | Gen T -> F T'
13 | $ E' T' ) E' T' F | id * id $ | Gen F -> id
14 | $ E' T' ) E' T' id | * id $ | Match id
15 | $ E' T' ) E' T' | * id $ | Gen T' -> * F T'
16 | $ E' T' ) E' T' F + | id $ | Match +
17 | $ E' T' ) E' T' F | id $ | Gen F -> id
18 | $ E' T' ) E' T' id | $ | Match id
19 | $ E' T' ) E' T' | $ | Gen T' -> epsilon
20 | $ E' T' ) E' | $ | Gen E' -> epsilon
21 | $ E' T' ) | $ | ERROR: Unexpected '$' Expected: ) Skipping '$'

Result: Parsing aborted with 1 error.

[Success] Parsing table safely saved to output/parsing_trace1.txt

TREE ERROR: Unexpected token $

===== PARSING TRACE 2 =====
Step | Stack | Input | Action
-----|-----|-----|-----
1 | $ E | id + * id $ | Gen E -> T E'
2 | $ E' T | id + * id $ | Gen T -> F T'
3 | $ E' T' F | id + * id $ | Gen F -> id
4 | $ E' T' id | + * id $ | Match id
5 | $ E' T' | + * id $ | Gen T' -> epsilon
6 | $ E' | + * id $ | Gen E' -> + T E'
7 | $ E' T + | * id $ | Match +
8 | $ E' T | * id $ | ERROR: No production for M[T, *]
9 | $ E' | * id $ | ERROR: No production for M[E', *]
10 | $ | * id $ | ERROR: Extra input remaining. Unexpected trailing '*'

Result: Parsing aborted with 3 error.

[Success] Parsing table safely saved to output/parsing_trace2.txt

```

```

TREE ERROR: No rule for (T, *)

===== PARSING TRACE 3 =====

Step | Stack | Input | Action
-----|-----|-----|-----
1 | $ E | id + $ | Gen E -> T E'
2 | $ E' T | id + $ | Gen T -> F T'
3 | $ E' T' F | id + $ | Gen F -> id
4 | $ E' T' id | + $ | Match id
5 | $ E' T' | + $ | Gen T' -> epsilon
6 | $ E' | + $ | Gen E' -> + T E'
7 | $ E' T + | $ | Match +
8 | $ E' T | $ | ERROR: No production for M[T, $]
9 | $ E' | $ | Gen E' -> epsilon
10 | $ | $ | Accept

Result: String accepted!

Result: Parsing completed with 1 error.

[Success] Parsing table safely saved to output/parsing_trace3.txt

TREE ERROR: No rule for (T, $)

===== PARSING TRACE 4 =====

Step | Stack | Input | Action
-----|-----|-----|-----
1 | $ E | id id $ | Gen E -> T E'
2 | $ E' T | id id $ | Gen T -> F T'
3 | $ E' T' F | id id $ | Gen F -> id
4 | $ E' T' id | id $ | Match id
5 | $ E' T' | id $ | ERROR: No production for M[T', id]
6 | $ E' | id $ | ERROR: No production for M[E', id]
7 | $ | id $ | ERROR: Extra input remaining. Unexpected trailing 'id'

Result: Parsing aborted with 3 error.

[Success] Parsing table safely saved to output/parsing_trace4.txt

TREE ERROR: No rule for (T', id)

```

```

===== PARSING TRACE 5 =====

Step | Stack | Input | Action
-----|-----|-----|-----
1 | $ E | * id + id $ | ERROR: No production for M[E, *]
2 | $ | * id + id $ | ERROR: Extra input remaining. Unexpected trailing '*'

Result: Parsing aborted with 2 error.

[Success] Parsing table safely saved to output/parsing_trace5.txt

TREE ERROR: No rule for (E, *)

===== PARSING TRACE 6 =====

Step | Stack | Input | Action
-----|-----|-----|-----
1 | $ E | ( ( id + id ) $ | Gen E -> T E'
2 | $ E' T | ( ( id + id ) $ | Gen T -> F T'
3 | $ E' T' F | ( ( id + id ) $ | Gen F -> ( E )
4 | $ E' T' ) E ( | ( id + id ) $ | Match (
5 | $ E' T' ) E | ( id + id ) $ | Gen E -> T E'
6 | $ E' T' ) E' T | ( id + id ) $ | Gen T -> F T'
7 | $ E' T' ) E' T' F | ( id + id ) $ | Gen F -> ( E )
8 | $ E' T' ) E' T' ) E ( | id + id ) $ | Match (
9 | $ E' T' ) E' T' ) E | id + id ) $ | Gen E -> T E'
10 | $ E' T' ) E' T' ) E' T | id + id ) $ | Gen T -> F T'
11 | $ E' T' ) E' T' ) E' T' F | id + id ) $ | Gen F -> id
12 | $ E' T' ) E' T' ) E' T' id | + id ) $ | Match id
13 | $ E' T' ) E' T' ) E' T' | + id ) $ | Gen T' -> epsilon
14 | $ E' T' ) E' T' ) E' | + id ) $ | Gen E' -> + T E'
15 | $ E' T' ) E' T' ) E' T + | id ) $ | Match +
16 | $ E' T' ) E' T' ) E' T | id ) $ | Gen T -> F T'
17 | $ E' T' ) E' T' ) E' T' F | id ) $ | Gen F -> id
18 | $ E' T' ) E' T' ) E' T' id | ) $ | Match id
19 | $ E' T' ) E' T' ) E' T' | ) $ | Gen T' -> epsilon
20 | $ E' T' ) E' T' ) E' | ) $ | Gen E' -> epsilon
21 | $ E' T' ) E' T' ) | $ | Match )
22 | $ E' T' ) E' T' | $ | Gen T' -> epsilon
23 | $ E' T' ) E' | $ | Gen E' -> epsilon
24 | $ E' T' ) | $ | ERROR: Unexpected '$' Expected: ) Skipping '$'

Result: Parsing aborted with 1 error.

[Success] Parsing table safely saved to output/parsing_trace6.txt

TREE ERROR: Unexpected token $

```

All Accept 1

```
=== Original Grammar ===
E -> T E'
E' -> + T E' | epsilon
T -> F T'
T' -> * F T' | epsilon
F -> ( E ) | id

=== Left Recursion Removed Grammar ===
E -> T E'
E' -> + T E' | epsilon
T -> F T'
T' -> * F T' | epsilon
F -> ( E ) | id

=== Left Factored Grammar ===
E -> T E'
E' -> + T E' | epsilon
T -> F T'
T' -> * F T' | epsilon
F -> ( E ) | id

[Success] Grammar safely saved to output/grammar_transformed.txt

=== First & Follow Sets ===
--- First() Sets ---
E:      { (, id }
E':     { +, epsilon }
T:      { (, id }
T':     { *, epsilon }
F:      { (, id }

--- Follow() Sets ---
E:      { $, ) }
E':     { $, ) }
T:      { $, ), + }
T':     { $, ), + }
F:      { $, ), *, + }

===== LL(1) PARSE TABLE =====
```

	+	*	(	)	id	\$
E	-	-	E->TE'	-	E->TE'	-
E'	E'->+TE'	-	-	E'->epsilon	-	E'->epsilon
T	-	-	T->FT'	-	T->FT'	-
T'	T'->epsilon	T'->*FT'	-	T'->epsilon	-	T'->epsilon
F	-	-	F->(E)	-	F->id	-

```
[Success] Parsing table safely saved to output/parsing_table.txt
```

===== PARSING TRACE 1 =====

Step	Stack	Input	Action
1	\$ E	id + (id * id) \$	Gen E -> T E'
2	\$ E' T	id + (id * id) \$	Gen T -> F T'
3	\$ E' T' F	id + (id * id) \$	Gen F -> id
4	\$ E' T' id	+ (id * id) \$	Match id
5	\$ E' T'	+ (id * id) \$	Gen T' -> epsilon
6	\$ E'	+ (id * id) \$	Gen E' -> + T E'
7	\$ E' T +	(id * id) \$	Match +
8	\$ E' T	(id * id) \$	Gen T -> F T'
9	\$ E' T' F	(id * id) \$	Gen F -> (E)
10	\$ E' T') E (	id * id) \$	Match (
11	\$ E' T') E	id * id) \$	Gen E -> T E'
12	\$ E' T') E' T	id * id) \$	Gen T -> F T'
13	\$ E' T') E' T' F	id * id) \$	Gen F -> id
14	\$ E' T') E' T' id	* id) \$	Match id
15	\$ E' T') E' T'	* id) \$	Gen T' -> * F T'
16	\$ E' T') E' T' F *	id) \$	Match *
17	\$ E' T') E' T' F	id) \$	Gen F -> id
18	\$ E' T') E' T' id	) \$	Match id
19	\$ E' T') E' T'	) \$	Gen T' -> epsilon
20	\$ E' T') E'	) \$	Gen E' -> epsilon
21	\$ E' T')	\$	Match)
22	\$ E' T'	\$	Gen T' -> epsilon
23	\$ E'	\$	Gen E' -> epsilon
24	\$	\$	Accept

Result: String accepted!

[Success] Parsing table safely saved to output/parsing_trace1.txt

``` ===== PARSE TREE (ASCII) ===== ```

```

\-- E
  |-- T
  |   |-- F
  |   |   \-- id
  |   |   \-- T'
  |   |       \-- epsilon
  |   \-- E'
  |       |-- +
  |       |-- T
  |       |   |-- F
  |       |   |   |-- (
  |       |   |   |-- E
  |       |   |   |   |-- T
  |       |   |   |   |   |-- F
  |       |   |   |   |   |   \-- id
  |       |   |   |   |   |   \-- T'
  |       |   |   |   |   |       |-- +
  |       |   |   |   |   |   |-- F
  |       |   |   |   |   |   |   \-- id
  |       |   |   |   |   |   |   \-- T'
  |       |   |   |   |   |       \-- epsilon
  |       |   |   |   |   \-- E'
  |       |   |   |   |       \-- epsilon
  |       |   |   |   \-- )
  |       |   |   \-- T'
  |       |   \-- epsilon
  |       \-- E'
  |           \-- epsilon
  
```

``` ===== PARSING TRACE 2 ===== ```

Step	Stack	Input	Action
1	\$ E	id + id * id \$	Gen E -> T E'
2	\$ E' T	id + id * id \$	Gen T -> F T'
3	\$ E' T' F	id + id * id \$	Gen F -> id
4	\$ E' T' id	+ id * id \$	Match id
5	\$ E' T'	+ id * id \$	Gen T' -> epsilon
6	\$ E'	+ id * id \$	Gen E' -> + T E'
7	\$ E' T +	id * id \$	Match +
8	\$ E' T	id * id \$	Gen T -> F T'
9	\$ E' T' F	id * id \$	Gen F -> id
10	\$ E' T' id	* id \$	Match id
11	\$ E' T'	* id \$	Gen T' -> * F T'
12	\$ E' T' F *	id \$	Match *
13	\$ E' T' F	id \$	Gen F -> id
14	\$ E' T' id	\$	Match id
15	\$ E' T'	\$	Gen T' -> epsilon
16	\$ E'	\$	Gen E' -> epsilon
17	\$	\$	Accept

Result: String accepted!

[Success] Parsing table safely saved to output/parsing_trace2.txt

===== PARSING TRACE 3 =====

Step	Stack	Input	Action
1	\$ E	id * id + id \$	Gen E -> T E'
2	\$ E' T	id * id + id \$	Gen T -> F T'
3	\$ E' T' F	id * id + id \$	Gen F -> id
4	\$ E' T' id	* id + id \$	Match id
5	\$ E' T'	* id + id \$	Gen T' -> * F T'
6	\$ E' T' F *	id + id \$	Match *
7	\$ E' T' F	id + id \$	Gen F -> id
8	\$ E' T' id	+ id \$	Match id
9	\$ E' T'	+ id \$	Gen T' -> epsilon
10	\$ E'	+ id \$	Gen E' -> + T E'
11	\$ E' T +	id \$	Match +
12	\$ E' T	id \$	Gen T -> F T'
13	\$ E' T' F	id \$	Gen F -> id
14	\$ E' T' id	\$	Match id
15	\$ E' T'	\$	Gen T' -> epsilon
16	\$ E'	\$	Gen E' -> epsilon
17	\$	\$	Accept

Result: String accepted!

[Success] Parsing table safely saved to output/parsing_trace3.txt

===== PARSE TREE (ASCII) =====

```

\-- E
    |-- T
    |   |-- F
    |   |   \-- id
    |   \-- T'
    |       |-- +
    |       |-- F
    |       |   \-- id
    |       \-- T'
    |           \-- epsilon
    \-- E'
        |-- +
        |-- T
        |   |-- F
        |   |   \-- id
        |   \-- T'
        |       \-- epsilon
        \-- E'
            \-- epsilon
  
```


===== PARSING TRACE 4 =====

Step	Stack	Input	Action
1	\$ E	(id * id) + id \$	Gen E -> T E'
2	\$ E' T	(id * id) + id \$	Gen T -> F T'
3	\$ E' T' F	(id * id) + id \$	Gen F -> (E)
4	\$ E' T') E (	id * id) + id \$	Match (
5	\$ E' T') E	id * id) + id \$	Gen E -> T E'
6	\$ E' T') E' T	id * id) + id \$	Gen T -> F T'
7	\$ E' T') E' T' F	id * id) + id \$	Gen F -> id
8	\$ E' T') E' T' id	* id) + id \$	Match id
9	\$ E' T') E' T'	* id) + id \$	Gen T' -> * F T'
10	\$ E' T') E' T' F *	id) + id \$	Match *
11	\$ E' T') E' T' F	id) + id \$	Gen F -> id
12	\$ E' T') E' T' id	) + id \$	Match id
13	\$ E' T') E' T'	) + id \$	Gen T' -> epsilon
14	\$ E' T') E'	) + id \$	Gen E' -> epsilon
15	\$ E' T')	+ id \$	Match)
16	\$ E' T'	+ id \$	Gen T' -> epsilon
17	\$ E'	+ id \$	Gen E' -> + T E'
18	\$ E' T +	id \$	Match +
19	\$ E' T	id \$	Gen T -> F T'
20	\$ E' T' F	id \$	Gen F -> id
21	\$ E' T' id	\$	Match id
22	\$ E' T'	\$	Gen T' -> epsilon
23	\$ E'	\$	Gen E' -> epsilon
24	\$	\$	Accept

Result: String accepted!

[Success] Parsing table safely saved to output/parsing_trace4.txt

```
===== PARSE TREE (ASCII) =====
```

```
\-- E
  |-- T
  |   |-- F
  |   |   |-- (
  |   |   |-- E
  |   |   |   |-- T
  |   |   |   |   |-- F
  |   |   |   |   |   |-- id
  |   |   |   |   |   |-- T'
  |   |   |   |   |   |-- +
  |   |   |   |   |   |-- F
  |   |   |   |   |   |   |-- id
  |   |   |   |   |   |   |-- T'
  |   |   |   |   |   |   |-- epsilon
  |   |   |   |   |-- E'
  |   |   |   |   |   |-- epsilon
  |   |   |   |-- )
  |   |   |-- T'
  |   |   |   |-- epsilon
  |-- E'
  |   |-- +
  |   |-- T
  |   |   |-- F
  |   |   |   |-- id
  |   |   |   |-- T'
  |   |   |   |-- epsilon
  |-- E'
  |   |-- epsilon
```

```
===== PARSING TRACE 5 =====
```

Step	Stack	Input	Action
1	\$ E	id * id \$	Gen E -> T E'
2	\$ E' T	id * id \$	Gen T -> F T'
3	\$ E' T' F	id * id \$	Gen F -> id
4	\$ E' T' id	* id \$	Match id
5	\$ E' T'	* id \$	Gen T' -> * F T'
6	\$ E' T' F *	id \$	Match *
7	\$ E' T' F	id \$	Gen F -> id
8	\$ E' T' id	\$	Match id
9	\$ E' T'	\$	Gen T' -> epsilon
10	\$ E'	\$	Gen E' -> epsilon
11	\$	\$	Accept

Result: String accepted!

[Success] Parsing table safely saved to output/parsing_trace5.txt

```
===== PARSE TREE (ASCII) =====
```

```
\-- E
  |-- T
  |   |-- F
  |   |   |-- id
  |   |-- T'
  |   |   |-- +
  |   |   |-- F
  |   |   |   |-- id
  |   |   |-- T'
  |   |   |   |-- epsilon
  |-- E'
  |   |-- epsilon
```

All Accept 2

```
=== Original Grammar ===
Expr -> SubExpr
SubExpr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> id | id ( Expr ) | ( Expr )

=== Left Recursion Removed Grammar ===
Expr -> Term NEWExpr
NEWExpr -> + Term NEWExpr | epsilon
SubExpr -> Expr + Term | Term
Term -> Factor NEWTerm
NEWTerm -> * Factor NEWTerm | epsilon
Factor -> id | id ( Expr ) | ( Expr )

=== Left Factored Grammar ===
Expr -> Term NEWExpr
NEWExpr -> + Term NEWExpr | epsilon
SubExpr -> Expr + Term | Term
Term -> Factor NEWTerm
NEWTerm -> * Factor NEWTerm | epsilon
Factor -> ( Expr ) | id Factor'
Factor' -> ( Expr ) | epsilon

[Success] Grammar safely saved to output/grammar_transformed.txt

=== First & Follow Sets ===
--- First() Sets ---
Expr: { (, id }
NEWExpr: { +, epsilon }
SubExpr: { (, id }
Term: { (, id }
NEWTerm: { *, epsilon }
Factor: { (, id }
Factor': { (, epsilon }

--- Follow() Sets ---
Expr: { $, ), + }
NEWExpr: { $, ), + }
SubExpr: { $, ), + }
Term: { $, ), + }
NEWTerm: { $, ), + }
Factor: { $, ), *, + }
Factor': { $, ), *, + }

Error
Error
Error

===== LL(1) PARSE TABLE =====
+      *      id      (      )      epsilon      $-----
Expr
pr->Term... - - - - - NEWExpr NEWExpr->+... - - - - - Expr->Term... Ex
- NEWExpr->e... SubExpr - - - - - SubExpr->E... SubExpr->E... - - - - - NEWExpr->e...
Term - - - - - Term->Fact... Term->Fact... - - - - - NEWTerm NEWTerm->e... NEWTerm-
>*... - - - - - NEWTerm->e... - - - - - NEWTerm->e... Factor - - - - - Factor->id... Factor->(E
... - - - - - Factor' Factor'->e... Factor'->e... - - - - - Factor'->(... Factor'->e...
- Factor'->e...

[Success] Parsing table safely saved to output/parsing_table.txt
```

```

===== PARSING TRACE 1 =====
Step | Stack | Input | Action
-----|-----|-----|-----
1 | $ Expr | id + id * id $ | Gen Expr -> Term NEWExpr
2 | $ NEWExpr Term | id + id * id $ | Gen Term -> Factor NEWTerm
3 | $ NEWExpr NEWTerm Factor | id + id * id $ | Gen Factor -> id Factor'
4 | $ NEWExpr NEWTerm Factor' id | + id * id $ | Match id
5 | $ NEWExpr NEWTerm Factor' | + id * id $ | Gen Factor' -> epsilon
6 | $ NEWExpr NEWTerm | + id * id $ | Gen NEWTerm -> epsilon
7 | $ NEWExpr | + id * id $ | Gen NEWExpr -> + Term NEWExpr
8 | $ NEWExpr Term + | id * id $ | Match +
9 | $ NEWExpr Term | id * id $ | Gen Term -> Factor NEWTerm
10 | $ NEWExpr NEWTerm Factor | id * id $ | Gen Factor -> id Factor'
11 | $ NEWExpr NEWTerm Factor' id | * id $ | Match id
12 | $ NEWExpr NEWTerm Factor' | * id $ | Gen Factor' -> epsilon
13 | $ NEWExpr NEWTerm | * id $ | Gen NEWTerm -> * Factor NEWTerm
14 | $ NEWExpr NEWTerm Factor * | id $ | Match *
15 | $ NEWExpr NEWTerm Factor | id $ | Gen Factor -> id Factor'
16 | $ NEWExpr NEWTerm Factor' id | $ | Match id
17 | $ NEWExpr NEWTerm Factor' | $ | Gen Factor' -> epsilon
18 | $ NEWExpr NEWTerm | $ | Gen NEWTerm -> epsilon
19 | $ NEWExpr | $ | Gen NEWExpr -> epsilon
20 | $ | $ | Accept

Result: String accepted!

[Success] Parsing table safely saved to output/parsing_trace1.txt

===== PARSE TREE (ASCII) =====
\-- Expr
  |-- Term
  |   |-- Factor
  |   |   |-- id
  |   |   \-- Factor'
  |   |       \-- epsilon
  |   \-- NEWTerm
  |       \-- epsilon
  \-- NEWExpr
      |-- +
      |-- Term
      |   |-- Factor
      |   |   |-- id
      |   |   \-- Factor'
      |   |       \-- epsilon
      |   \-- NEWTerm
      |       |-- *
      |       |-- Factor
      |       |   |-- id
      |       |   \-- Factor'
      |       |       \-- epsilon
      |       \-- NEWTerm
      |           \-- epsilon
      \-- NEWExpr
          \-- epsilon

```

```

===== PARSING TRACE 2 =====
Step | Stack | Input | Action
-----|-----|-----|-----
1 | $ Expr | ( id + id ) * id $ | Gen Expr -> Term NEWExpr
2 | $ NEWExpr Term | ( id + id ) * id $ | Gen Term -> Factor NEWTerm
3 | $ NEWExpr NEWTerm Factor | ( id + id ) * id $ | Gen Factor -> ( Expr )
4 | $ NEWExpr NEWTerm ) Expr ( | id + id ) * id $ | Match (
5 | $ NEWExpr NEWTerm ) Expr | id + id ) * id $ | Gen Expr -> Term NEWExpr
6 | $ NEWExpr NEWTerm ) NEWExpr Term | id + id ) * id $ | Gen Term -> Factor NEWTerm
7 | $ NEWExpr NEWTerm ) NEWExpr NEWTerm Factor | id + id ) * id $ | Gen Factor -> id Factor'
8 | $ NEWExpr NEWTerm ) NEWExpr NEWTerm Factor' id | + id ) * id $ | Match id
9 | $ NEWExpr NEWTerm ) NEWExpr NEWTerm Factor' | + id ) * id $ | Gen Factor' -> epsilon
10 | $ NEWExpr NEWTerm ) NEWExpr NEWTerm | + id ) * id $ | Gen NEWTerm -> epsilon
11 | $ NEWExpr NEWTerm ) NEWExpr | + id ) * id $ | Gen NEWExpr -> + Term NEWExpr
12 | $ NEWExpr NEWTerm ) NEWExpr Term + | id ) * id $ | Match +
13 | $ NEWExpr NEWTerm ) NEWExpr Term | id ) * id $ | Gen Term -> Factor NEWTerm
14 | $ NEWExpr NEWTerm ) NEWExpr NEWTerm Factor | id ) * id $ | Gen Factor -> id Factor'
15 | $ NEWExpr NEWTerm ) NEWExpr NEWTerm Factor' id | ) * id $ | Match id
16 | $ NEWExpr NEWTerm ) NEWExpr NEWTerm Factor' | ) * id $ | Gen Factor' -> epsilon
17 | $ NEWExpr NEWTerm ) NEWExpr NEWTerm | ) * id $ | Gen NEWTerm -> epsilon
18 | $ NEWExpr NEWTerm ) NEWExpr | ) * id $ | Gen NEWExpr -> epsilon
19 | $ NEWExpr NEWTerm ) | * id $ | Match )
20 | $ NEWExpr NEWTerm | * id $ | Gen NEWTerm -> * Factor NEWTerm
21 | $ NEWExpr NEWTerm Factor * | id $ | Match *
22 | $ NEWExpr NEWTerm Factor | id $ | Gen Factor -> id Factor'
23 | $ NEWExpr NEWTerm Factor' id | $ | Match id
24 | $ NEWExpr NEWTerm Factor' | $ | Gen Factor' -> epsilon
25 | $ NEWExpr NEWTerm | $ | Gen NEWTerm -> epsilon
26 | $ NEWExpr | $ | Gen NEWExpr -> epsilon
27 | $ | $ | Accept

Result: String accepted!

[Success] Parsing table safely saved to output/parsing_trace2.txt

```

``` ===== PARSE TREE (ASCII) ===== ```

```

\-- Expr
  |-- Term
  |   |-- Factor
  |   |   |-- (
  |   |   |-- Expr
  |   |   |   |-- Term
  |   |   |   |   |-- Factor
  |   |   |   |   |   |-- id
  |   |   |   |   |   \-- Factor'
  |   |   |   |   |       \-- epsilon
  |   |   |   |   \-- NEWTerm
  |   |   |   |       \-- epsilon
  |   |   |   \-- NEWExpr
  |   |   |       |-- +
  |   |   |       |-- Term
  |   |   |       |   |-- Factor
  |   |   |       |   |   |-- id
  |   |   |       |   |   \-- Factor'
  |   |   |       |   |       \-- epsilon
  |   |   |       |   \-- NEWTerm
  |   |   |       |       \-- epsilon
  |   |   |       \-- NEWExpr
  |   |   |           \-- epsilon
  |   |   \-- )
  |   \-- NEWTerm
  |       |-- *
  |       |-- Factor
  |       |   |-- id
  |       |   \-- Factor'
  |       |       \-- epsilon
  |       \-- NEWTerm
  |           \-- epsilon
  \-- NEWExpr
      \-- epsilon

```

``` ===== PARSING TRACE 3 ===== ```

Step	Stack	Input	Action
1	\$ Expr	id * id + id \$	Gen Expr -> Term NEWExpr
2	\$ NEWExpr Term	id * id + id \$	Gen Term -> Factor NEWTerm
3	\$ NEWExpr NEWTerm Factor	id * id + id \$	Gen Factor -> id Factor'
4	\$ NEWExpr NEWTerm Factor' id	* id + id \$	Match id
5	\$ NEWExpr NEWTerm Factor'	* id + id \$	Gen Factor' -> epsilon
6	\$ NEWExpr NEWTerm	* id + id \$	Gen NEWTerm -> * Factor NEWTerm
7	\$ NEWExpr NEWTerm Factor *	id + id \$	Match *
8	\$ NEWExpr NEWTerm Factor	id + id \$	Gen Factor -> id Factor'
9	\$ NEWExpr NEWTerm Factor' id	+ id \$	Match id
10	\$ NEWExpr NEWTerm Factor'	+ id \$	Gen Factor' -> epsilon
11	\$ NEWExpr NEWTerm	+ id \$	Gen NEWTerm -> epsilon
12	\$ NEWExpr	+ id \$	Gen NEWExpr -> + Term NEWExpr
13	\$ NEWExpr Term +	id \$	Match +
14	\$ NEWExpr Term	id \$	Gen Term -> Factor NEWTerm
15	\$ NEWExpr NEWTerm Factor	id \$	Gen Factor -> id Factor'
16	\$ NEWExpr NEWTerm Factor' id	\$	Match id
17	\$ NEWExpr NEWTerm Factor'	\$	Gen Factor' -> epsilon
18	\$ NEWExpr NEWTerm	\$	Gen NEWTerm -> epsilon
19	\$ NEWExpr	\$	Gen NEWExpr -> epsilon
20	\$	\$	Accept

Result: String accepted!

[Success] Parsing table safely saved to output/parsing_trace3.txt

```

===== PARSE TREE (ASCII) =====
\-- Expr
|-- Term
|   |-- Factor
|   |   |-- id
|   |   \-- Factor'
|   |       \-- epsilon
|   \-- NEWTerm
|       |-- *
|       |-- Factor
|       |   |-- id
|       |   \-- Factor'
|       |       \-- epsilon
|       \-- NEWTerm
|           \-- epsilon
\-- NEWExpr
    |-- +
    |-- Term
    |   |-- Factor
    |   |   |-- id
    |   |   \-- Factor'
    |   |       \-- epsilon
    |   \-- NEWTerm
    |       \-- epsilon
    \-- NEWExpr
        \-- epsilon

===== PARSING TRACE 4 =====

```

Step	Stack	Input	Action
1	\$ Expr	id * (id + id) \$	Gen Expr -> Term NEWExpr
2	\$ NEWExpr Term	id * (id + id) \$	Gen Term -> Factor NEWTerm
3	\$ NEWExpr NEWTerm Factor	id * (id + id) \$	Gen Factor -> id Factor'
4	\$ NEWExpr NEWTerm Factor' id	* (id + id) \$	Match id
5	\$ NEWExpr NEWTerm Factor'	* (id + id) \$	Gen Factor' -> epsilon
6	\$ NEWExpr NEWTerm	* (id + id) \$	Gen NEWTerm -> * Factor NEWTerm
7	\$ NEWExpr NEWTerm Factor *	(id + id) \$	Match *
8	\$ NEWExpr NEWTerm Factor	(id + id) \$	Gen Factor -> (Expr)
9	\$ NEWExpr NEWTerm) Expr (	id + id) \$	Match (
10	\$ NEWExpr NEWTerm) Expr	id + id) \$	Gen Expr -> Term NEWExpr
11	\$ NEWExpr NEWTerm) NEWExpr Term	id + id) \$	Gen Term -> Factor NEWTerm
12	\$ NEWExpr NEWTerm) NEWExpr NEWTerm Factor	id + id) \$	Gen Factor -> id Factor'
13	\$ NEWExpr NEWTerm) NEWExpr NEWTerm Factor' id	+ id) \$	Match id
14	\$ NEWExpr NEWTerm) NEWExpr NEWTerm Factor'	+ id) \$	Gen Factor' -> epsilon
15	\$ NEWExpr NEWTerm) NEWExpr NEWTerm	+ id) \$	Gen NEWTerm -> epsilon
16	\$ NEWExpr NEWTerm) NEWExpr	+ id) \$	Gen NEWExpr -> + Term NEWExpr
17	\$ NEWExpr NEWTerm) NEWExpr Term +	id) \$	Match +
18	\$ NEWExpr NEWTerm) NEWExpr Term	id) \$	Gen Term -> Factor NEWTerm
19	\$ NEWExpr NEWTerm) NEWExpr NEWTerm Factor	id) \$	Gen Factor -> id Factor'
20	\$ NEWExpr NEWTerm) NEWExpr NEWTerm Factor' id	) \$	Match id
21	\$ NEWExpr NEWTerm) NEWExpr NEWTerm Factor'	) \$	Gen Factor' -> epsilon
22	\$ NEWExpr NEWTerm) NEWExpr NEWTerm	) \$	Gen NEWTerm -> epsilon
23	\$ NEWExpr NEWTerm) NEWExpr	) \$	Gen NEWExpr -> epsilon
24	\$ NEWExpr NEWTerm)	\$	Match)
25	\$ NEWExpr NEWTerm	\$	Gen NEWTerm -> epsilon
26	\$ NEWExpr	\$	Gen NEWExpr -> epsilon
27	\$	\$	Accept

Result: String accepted!

[Success] Parsing table safely saved to output/parsing_trace4.txt

```

===== PARSE TREE (ASCII) =====
\-- Expr
|-- Term
|   |-- Factor
|   |   |-- id
|   |   \-- Factor'
|   |       \-- epsilon
|   \-- NEWTerm
|       |-- *
|       |-- Factor
|       |   |-- (
|       |   |-- Expr
|       |   |   |-- Term
|       |   |   |   |-- Factor
|       |   |   |   |   |-- id
|       |   |   |   |   \-- Factor'
|       |   |   |   |       \-- epsilon
|       |   |   |   \-- NEWTerm
|       |   |   |       \-- epsilon
|       |   |   \-- NEWExpr
|       |   |       |-- +
|       |   |       |-- Term
|       |   |       |   |-- Factor
|       |   |       |   |   |-- id
|       |   |       |   |   \-- Factor'
|       |   |       |   |       \-- epsilon
|       |   |       |   \-- NEWTerm
|       |   |       |       \-- epsilon
|       |   |       \-- NEWExpr
|       |   |           \-- epsilon
|       |   \-- )
|       \-- NEWTerm
|           \-- epsilon
\-- NEWExpr
    \-- epsilon

===== PARSING TRACE 5 =====

```

Step	Stack	Input	Action
1	\$ Expr	id + id \$	Gen Expr -> Term NEWExpr
2	\$ NEWExpr Term	id + id \$	Gen Term -> Factor NEWTerm
3	\$ NEWExpr NEWTerm Factor	id + id \$	Gen Factor -> id Factor'
4	\$ NEWExpr NEWTerm Factor' id	+ id \$	Match id
5	\$ NEWExpr NEWTerm Factor'	+ id \$	Gen Factor' -> epsilon
6	\$ NEWExpr NEWTerm	+ id \$	Gen NEWTerm -> epsilon
7	\$ NEWExpr	+ id \$	Gen NEWExpr -> + Term NEWExpr
8	\$ NEWExpr Term +	id \$	Match +
9	\$ NEWExpr Term	id \$	Gen Term -> Factor NEWTerm
10	\$ NEWExpr NEWTerm Factor	id \$	Gen Factor -> id Factor'
11	\$ NEWExpr NEWTerm Factor' id	\$	Match id
12	\$ NEWExpr NEWTerm Factor'	\$	Gen Factor' -> epsilon
13	\$ NEWExpr NEWTerm	\$	Gen NEWTerm -> epsilon
14	\$ NEWExpr	\$	Gen NEWExpr -> epsilon
15	\$	\$	Accept

Result: String accepted!

[Success] Parsing table safely saved to output/parsing_trace5.txt


```
===== PARSE TREE (ASCII) =====
```

```
\-- Expr
  |-- Term
  |   |-- Factor
  |   |   |-- id
  |   |   \-- Factor'
  |   |       \-- epsilon
  |   \-- NEWTerm
  |       \-- epsilon
  \-- NEWExpr
      |-- +
      |-- Term
      |   |-- Factor
      |   |   |-- id
      |   |   \-- Factor'
      |   |       \-- epsilon
      |   \-- NEWTerm
      |       \-- epsilon
      \-- NEWExpr
          \-- epsilon
```

Conclusion

Through this LL(1) parsing assignment, we learned the complete process of transforming a context-free grammar into a form suitable for predictive parsing. We gained hands-on experience in eliminating both direct and indirect left recursion, which is essential to prevent infinite recursion in top-down parsers. We also learned the importance of left factoring to remove common prefixes, enabling the parser to make decisions using a single lookahead symbol. The assignment reinforced the computation of FIRST and FOLLOW sets, highlighting how epsilon productions affect parsing decisions and table construction. Building the LL(1) parsing table demonstrated how these sets are applied in practice to map non-terminals and lookahead symbols to appropriate productions, while also teaching us to detect conflicts that indicate non-LL(1) grammars. Finally, implementing the parse tree and its visualization helped us understand the derivation process of input strings, reinforcing the connection between grammar rules and the parsing mechanism. Overall, the assignment provided a thorough understanding of predictive parsing, grammar preprocessing, and the practical considerations in designing an LL(1) parser.

GitHub Private Repository Link

<https://github.com/muhammad-moiz-ansari/MA---Compiler>